

MANUAL TÉCNICO

Proyecto 1 Sistemas Operativos 1

Universidad de San Carlos de Guatemala

Estudiante: Tulio Jafeth Pirir Schuman

Carnet: 201700698

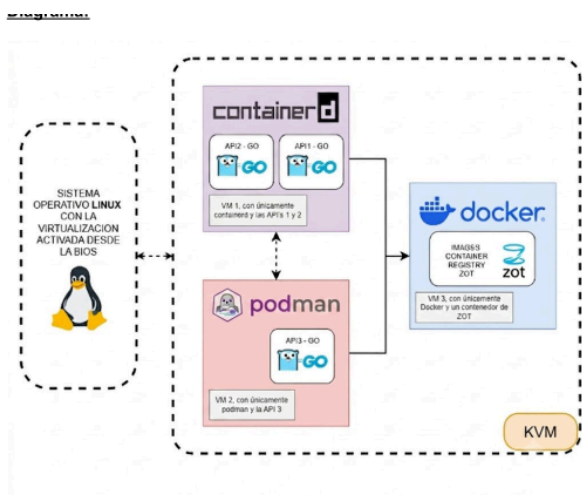
1. INTRODUCCIÓN Y

El presente manual detalla la infraestructura física y lógica implementada para el proyecto. La solución se basa en KVM (Kernel-based Virtual Machine), una tecnología de virtualización completa integrada en el kernel Linux que convierte al sistema operativo host en un hipervisor de Tipo 1 (Bare-Metal).

KVM utiliza el módulo del kernel `kvm.ko` para exponer la interfaz `/dev/kvm`, permitiendo que el emulador de hardware QEMU (Quick Emulator) ejecute instrucciones directamente en la CPU del servidor host mediante extensiones de hardware (Intel VT-x o AMD-V). La gestión de las máquinas virtuales se canaliza a través de la API libvirt y el conjunto de herramientas de línea de comandos `virsh`.

2. ARQUITECTURA DETALLADA DEL SISTEMA Y RED VIRTUAL

La infraestructura está compuesta por tres máquinas virtuales (Nodos) conectadas a un puente de red virtualizado (`virbr0`) configurado por `libvirtd` bajo la subred `192.168.122.0/24`



Nodos e Infraestructura

- VM1 (192.168.122.11): Node Containerd / nerdctl
 - OS: Ubuntu Server / Debian Linux.
 - Runtime de Contenedores: containerd controlado vía nerdctl.
 - Servicios: API1 (:8081) y API2 (:8082).
 - Estrategia de Red: Control de interfaz compartida (--net=host) para acoplamiento de comunicación interna sobre la interfaz loopback local (127.0.0.1).
- VM2 (192.168.122.12): Node Podman Daemonless
 - OS: Ubuntu Server / Debian Linux.
 - Runtime de Contenedores: Podman (Motor daemonless sin demonio persistente en segundo plano).
 - Servicios: API3 (:8083).
 - Estrategia de Red: Red virtual aislada tipo puente con reenvío de puertos (Port Forwarding) -p 8083:8083.
- VM3 (192.168.122.13): Node Central OCI Registry
 - OS: Ubuntu Server / Debian Linux.
 - Runtime de Contenedores: Docker Engine.
 - Servicios: Zot OCI Registry (:5000) respaldado por un volumen persistente montado en el almacenamiento local de la VM.

3. RUNTIMES, HERRAMIENTAS Y CONTROLADORES DE RED

Tecnologías y Runtimes Seleccionados

1. containerd & nerdctl: Runtime de contenedores impulsado por la CNCF (Cloud Native Computing Foundation). Administra el almacenamiento, la transferencia de imágenes OCI y la ejecución de bajo nivel invocando a runc. Se utiliza la CLI nerdctl como interfaz compatible con la sintaxis de Docker.
2. Podman: Motor de ejecución que sigue el modelo OCI sin requerir un proceso centralizado (daemon). Aumenta la seguridad al ejecutar contenedores en modo rootless aprovechando los User Namespaces del kernel Linux.
3. Docker Engine: Utilizado en VM3 para mantener la estabilidad del servicio Zot, un registro de imágenes ligero, seguro y compatible con las especificaciones de distribución OCI V2.
4. jq & curl: curl gestiona la emisión de peticiones HTTP en la capa de aplicación, mientras que jq actúa como procesador de flujos JSON para formatear y validar los payloads en la terminal.

3.2 Análisis de Red: Host Driver (--net=host) vs Bridge Driver

- Bridge (Puente Virtual): Crea una interfaz de red virtual (p. ej., cni0) y asigna una subred privada a los contenedores. Para comunicarse externamente, el kernel aplica reglas de NAT con iptables / nftables. Esto causa un bloqueo al intentar redirigir tráfico interno entre dos contenedores locales que intentan acceder a la dirección loopback del host.

- Host Driver (--net=host): Remueve el espacio de nombres de red (network namespace) del contenedor. Los sockets del contenedor se enlazan directamente a la pila TCP/IP del sistema operativo de la máquina virtual. Esto permite que la API1 llame directamente a la API2 mediante 127.0.0.1:8082 en VM1 sin requerir reglas de enrutamiento o filtrado intermedio.

4. ENDPOINTS

Origen	Endpoint Invocado	Destino	Tipo de Conexión
VM1	GET /health	API1 (:8081)	Estado local de la API1.
VM1	GET /health	API2 (:8082)	Estado local de la API2.
VM2	GET /health	API3 (:8083)	Estado local de la API3.
VM1	GET /api1/201700698/call -api2	API2 (:8082)	Intra-VM (Localhost via --net=host).
VM1	GET /api1/201700698/call -api3	API3 (192.168.122.12:8083)	Inter-VM (VM1 -> VM2).
VM1	GET /api2/201700698/call -api1	API1 (:8081)	Intra-VM (Localhost via --net=host).
VM1	GET /api2/201700698/call -api3	API3 (192.168.122.12:8083)	Inter-VM (VM1 -> VM2).

VM2	GET /api3/201700698/call -api1	API1 (192.168.122.11:8081)	Inter-VM (VM2 -> VM1).
VM2	GET /api3/201700698/call -api2	API2 (192.168.122.11:8082)	Inter-VM (VM2 -> VM1).

5. MECANISMO DE RESILIENCIA Y MANEJO DE ERRORES EN GO

Las aplicaciones están escritas en Go utilizando net/http con un cliente configurado con tiempos límite (timeouts) estrictos.

Go

```
client := &http.Client{
    Timeout: 3 * time.Second,
}
```

```
resp, err := client.Get(targetURL)
```

```
if err != nil {
```

```
    // Si la API destino no responde, se evita la caída del servidor
```

```
    w.Header().Set("Content-Type", "application/json")
```

```
    json.NewEncoder(w).Encode(map[string]interface{}{
```

```
        "apiname": targetAPIName,
```

```
        "message": fmt.Sprintf("ERROR: The %s located on destination VM is not  
working", targetAPIName),
```

```
        "connection": false,
```

```
        "carnet": "201700698",
```

```
    })
```

```
    return
```

```
}
```

```
defer resp.Body.Close()
```